

Integrate the <SplashCursor /> component from React Bits

You are helping integrate an open-source React component into an existing application.

Component: SplashCursor

Variant: JavaScript + CSS

Usage Example

```
```jsx
```

```
import SplashCursor from './SplashCursor'
```

```
<SplashCursor />
```

```
```
```

Props

| Prop | Type | Default | Description |
|------|------|---------|-------------|
|------|------|---------|-------------|

| | | | |
|----------------|--------|-----|--|
| SIM_RESOLUTION | number | 128 | Fluid simulation resolution for velocity fields. |
|----------------|--------|-----|--|

| | | | |
|----------------|--------|------|--------------------------------------|
| DYE_RESOLUTION | number | 1440 | Resolution of the color/dye texture. |
|----------------|--------|------|--------------------------------------|

| | | | |
|--------------------|--------|-----|--|
| CAPTURE_RESOLUTION | number | 512 | Resolution used for certain capture operations (rarely changed). |
|--------------------|--------|-----|--|

| | | | |
|---------------------|--------|-----|---|
| DENSITY DISSIPATION | number | 3.5 | Rate at which color/density dissipates over time. |
|---------------------|--------|-----|---|

| | | | |
|----------------------|--------|---|--|
| VELOCITY DISSIPATION | number | 2 | Rate at which velocity dissipates over time. |
|----------------------|--------|---|--|

| | | | |
|----------|--------|-----|---|
| PRESSURE | number | 0.1 | Base pressure for the fluid simulation. |
|----------|--------|-----|---|

| | | | |
|---------------------|--------|----|---|
| PRESSURE_ITERATIONS | number | 20 | Number of Jacobi iterations used for the pressure solver. |
|---------------------|--------|----|---|

| | | | |
|------|--------|---|---|
| CURL | number | 3 | Amount of vorticity/curl to apply for swirling effects. |
|------|--------|---|---|

| | | | |
|--------------|--------|-----|---|
| SPLAT_RADIUS | number | 0.2 | Radius of the 'splat' effect when user interacts. |
|--------------|--------|-----|---|

| | | | |
|-------------|--------|------|---|
| SPLAT_FORCE | number | 6000 | Force of the fluid 'splat' on each interaction. |
|-------------|--------|------|---|

| | | | |
|---------|---------|------|---|
| SHADING | boolean | true | Toggles simple lighting/shading on the fluid. |
|---------|---------|------|---|

| | | | |
|--------------------|--------|----|--|
| COLOR_UPDATE_SPEED | number | 10 | Frequency at which pointer colors are re-randomized. |
|--------------------|--------|----|--|

| | | | |
|--------------|---------|------|---|
| RAINBOW_MODE | boolean | true | When true, uses randomly cycling rainbow colors. When false, uses the COLOR prop. |
|--------------|---------|------|---|

| | | | |
|-------|--------|-----------|---|
| COLOR | string | '#ff0000' | Hex color for the cursor effect when RAINBOW_MODE is false. |
|-------|--------|-----------|---|

| | | | |
|------------|--------|------------------------|--|
| BACK_COLOR | object | { r: 0.5, g: 0, b: 0 } | Base background color for the fluid. Not always used if TRANSPARENT is true. |
|------------|--------|------------------------|--|

Full Component Source

```
```jsx
```

```
'use client';
```

```

import { useEffect, useRef } from 'react';

function SplashCursor({
 SIM_RESOLUTION = 128,
 DYE_RESOLUTION = 1440,
 CAPTURE_RESOLUTION = 512,
 DENSITY DISSIPATION = 3.5,
 VELOCITY DISSIPATION = 2,
 PRESSURE = 0.1,
 PRESSURE_ITERATIONS = 20,
 CURL = 3,
 SPLAT_RADIUS = 0.2,
 SPLAT_FORCE = 6000,
 SHADING = true,
 COLOR_UPDATE_SPEED = 10,
 BACK_COLOR = { r: 0.5, g: 0, b: 0 },
 TRANSPARENT = true,
 RAINBOW_MODE = true,
 COLOR = '#ff0000'
}) {
 const canvasRef = useRef(null);
 const animationFrameId = useRef(null);

 useEffect(() => {
 const canvas = canvasRef.current;
 if (!canvas) return;

 // Track if the effect is still active for cleanup
 let isActive = true;

 function pointerPrototype() {
 this.id = -1;
 this.texcoordX = 0;
 this.texcoordY = 0;
 this.prevTexcoordX = 0;
 this.prevTexcoordY = 0;
 this.deltaX = 0;
 this.deltaY = 0;
 this.down = false;
 this.moved = false;
 this.color = [0, 0, 0];
 }

 let config = {

```

```
SIM_RESOLUTION,
DYE_RESOLUTION,
CAPTURE_RESOLUTION,
DENSITY_DISSIPATION,
VELOCITY_DISSIPATION,
PRESSURE,
PRESSURE_ITERATIONS,
CURL,
SPLAT_RADIUS,
SPLAT_FORCE,
SHADING,
COLOR_UPDATE_SPEED,
PAUSED: false,
BACK_COLOR,
TRANSPARENT,
RAINBOW_MODE,
COLOR
};
```

```
let pointers = [new pointerPrototype()];
```

```
const { gl, ext } = getWebGLContext(canvas);
if (!ext.supportLinearFiltering) {
 config.DYE_RESOLUTION = 256;
 config.SHADING = false;
}
```

```
function getWebGLContext(canvas) {
 const params = {
 alpha: true,
 depth: false,
 stencil: false,
 antialias: false,
 preserveDrawingBuffer: false
 };
 let gl = canvas.getContext('webgl2', params);
 const isWebGL2 = !!gl;
 if (!isWebGL2) gl = canvas.getContext('webgl', params) ||
 canvas.getContext('experimental-webgl', params);

 let halfFloat;
 let supportLinearFiltering;
 if (isWebGL2) {
 gl.getExtension('EXT_color_buffer_float');
```

```

 supportLinearFiltering = gl.getExtension('OES_texture_float_linear');
} else {
 halfFloat = gl.getExtension('OES_texture_half_float');
 supportLinearFiltering = gl.getExtension('OES_texture_half_float_linear');
}
gl.clearColor(0.0, 0.0, 0.0, 1.0);

const halfFloatTexType = isWebGL2 ? gl.HALF_FLOAT : halfFloat &&
halfFloat.HALF_FLOAT_OES;
let formatRGBA;
let formatRG;
let formatR;

if (isWebGL2) {
 formatRGBA = getSupportedFormat(gl, gl.RGBA16F, gl.RGBA, halfFloatTexType);
 formatRG = getSupportedFormat(gl, gl.RG16F, gl.RG, halfFloatTexType);
 formatR = getSupportedFormat(gl, gl.R16F, gl.RED, halfFloatTexType);
} else {
 formatRGBA = getSupportedFormat(gl, gl.RGBA, gl.RGBA, halfFloatTexType);
 formatRG = getSupportedFormat(gl, gl.RGBA, gl.RGBA, halfFloatTexType);
 formatR = getSupportedFormat(gl, gl.RGBA, gl.RGBA, halfFloatTexType);
}

return {
 gl,
 ext: {
 formatRGBA,
 formatRG,
 formatR,
 halfFloatTexType,
 supportLinearFiltering
 }
};
}

function getSupportedFormat(gl, internalFormat, format, type) {
 if (!supportRenderTextureFormat(gl, internalFormat, format, type)) {
 switch (internalFormat) {
 case gl.R16F:
 return getSupportedFormat(gl, gl.RG16F, gl.RG, type);
 case gl.RG16F:
 return getSupportedFormat(gl, gl.RGBA16F, gl.RGBA, type);
 default:
 return null;
 }
 }
}

```

```

 }
 }
 return { internalFormat, format };
}

function supportRenderTextureFormat(gl, internalFormat, format, type) {
 const texture = gl.createTexture();
 gl.bindTexture(gl.TEXTURE_2D, texture);
 gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_MIN_FILTER, gl.NEAREST);
 gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_MAG_FILTER, gl.NEAREST);
 gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_WRAP_S, gl.CLAMP_TO_EDGE);
 gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_WRAP_T, gl.CLAMP_TO_EDGE);
 gl.texImage2D(gl.TEXTURE_2D, 0, internalFormat, 4, 4, 0, format, type, null);
 const fbo = gl.createFramebuffer();
 gl.bindFramebuffer(gl.FRAMEBUFFER, fbo);
 gl.framebufferTexture2D(gl.FRAMEBUFFER, gl.COLOR_ATTACHMENT0,
gl.TEXTURE_2D, texture, 0);
 const status = gl.checkFramebufferStatus(gl.FRAMEBUFFER);
 return status === gl.FRAMEBUFFER_COMPLETE;
}

class Material {
 constructor(vertexShader, fragmentShaderSource) {
 this.vertexShader = vertexShader;
 this.fragmentShaderSource = fragmentShaderSource;
 this.programs = [];
 this.activeProgram = null;
 this.uniforms = [];
 }
 setKeywords(keywords) {
 let hash = 0;
 for (let i = 0; i < keywords.length; i++) hash += hashCode(keywords[i]);
 let program = this.programs[hash];
 if (program == null) {
 let fragmentShader = compileShader(gl.FRAGMENT_SHADER,
this.fragmentShaderSource, keywords);
 program = createProgram(this.vertexShader, fragmentShader);
 this.programs[hash] = program;
 }
 if (program === this.activeProgram) return;
 this.uniforms = getUniforms(program);
 this.activeProgram = program;
 }
 bind() {

```

```

 gl.useProgram(this.activeProgram);
 }
}

class Program {
 constructor(vertexShader, fragmentShader) {
 this.uniforms = {};
 this.program = createProgram(vertexShader, fragmentShader);
 this.uniforms = getUniforms(this.program);
 }
 bind() {
 gl.useProgram(this.program);
 }
}

function createProgram(vertexShader, fragmentShader) {
 let program = gl.createProgram();
 gl.attachShader(program, vertexShader);
 gl.attachShader(program, fragmentShader);
 gl.linkProgram(program);
 if (!gl.getProgramParameter(program, gl.LINK_STATUS))
 console.trace(gl.getProgramInfoLog(program));
 return program;
}

function getUniforms(program) {
 let uniforms = [];
 let uniformCount = gl.getProgramParameter(program, gl.ACTIVE_UNIFORMS);
 for (let i = 0; i < uniformCount; i++) {
 let uniformName = gl.getActiveUniform(program, i).name;
 uniforms[uniformName] = gl.getUniformLocation(program, uniformName);
 }
 return uniforms;
}

function compileShader(type, source, keywords) {
 source = addKeywords(source, keywords);
 const shader = gl.createShader(type);
 gl.shaderSource(shader, source);
 gl.compileShader(shader);
 if (!gl.getShaderParameter(shader, gl.COMPILE_STATUS))
 console.trace(gl.getShaderInfoLog(shader));
 return shader;
}

```

```

function addKeywords(source, keywords) {
 if (!keywords) return source;
 let keywordsString = "";
 keywords.forEach(keyword => {
 keywordsString += '#define ' + keyword + '\n';
 });
 return keywordsString + source;
}

```

```

const baseVertexShader = compileShader(
 gl.VERTEX_SHADER,
 `
 precision highp float;
 attribute vec2 aPosition;
 varying vec2 vUv;
 varying vec2 vL;
 varying vec2 vR;
 varying vec2 vT;
 varying vec2 vB;
 uniform vec2 texelSize;

 void main () {
 vUv = aPosition * 0.5 + 0.5;
 vL = vUv - vec2(texelSize.x, 0.0);
 vR = vUv + vec2(texelSize.x, 0.0);
 vT = vUv + vec2(0.0, texelSize.y);
 vB = vUv - vec2(0.0, texelSize.y);
 gl_Position = vec4(aPosition, 0.0, 1.0);
 }
 `
);

```

```

const copyShader = compileShader(
 gl.FRAGMENT_SHADER,
 `
 precision mediump float;
 precision mediump sampler2D;
 varying highp vec2 vUv;
 uniform sampler2D uTexture;

 void main () {
 gl_FragColor = texture2D(uTexture, vUv);
 }
 `
);

```

```
);
```

```
const clearShader = compileShader(
 gl.FRAGMENT_SHADER,
 `
 precision mediump float;
 precision mediump sampler2D;
 varying highp vec2 vUv;
 uniform sampler2D uTexture;
 uniform float value;

 void main () {
 gl_FragColor = value * texture2D(uTexture, vUv);
 }
 `
);
```

```
const displayShaderSource = `
 precision highp float;
 precision highp sampler2D;
 varying vec2 vUv;
 varying vec2 vL;
 varying vec2 vR;
 varying vec2 vT;
 varying vec2 vB;
 uniform sampler2D uTexture;
 uniform sampler2D uDithering;
 uniform vec2 ditherScale;
 uniform vec2 texelSize;

 vec3 linearToGamma (vec3 color) {
 color = max(color, vec3(0));
 return max(1.055 * pow(color, vec3(0.416666667)) - 0.055, vec3(0));
 }

 void main () {
 vec3 c = texture2D(uTexture, vUv).rgb;
 #ifdef SHADING
 vec3 lc = texture2D(uTexture, vL).rgb;
 vec3 rc = texture2D(uTexture, vR).rgb;
 vec3 tc = texture2D(uTexture, vT).rgb;
 vec3 bc = texture2D(uTexture, vB).rgb;
```

```

float dx = length(rc) - length(lc);
float dy = length(tc) - length(bc);

vec3 n = normalize(vec3(dx, dy, length(texelSize)));
vec3 l = vec3(0.0, 0.0, 1.0);

float diffuse = clamp(dot(n, l) + 0.7, 0.7, 1.0);
c *= diffuse;
#endif

float a = max(c.r, max(c.g, c.b));
gl_FragColor = vec4(c, a);
}
};

```

```

const splatShader = compileShader(
 gl.FRAGMENT_SHADER,
 ,
 precision highp float;
 precision highp sampler2D;
 varying vec2 vUv;
 uniform sampler2D uTarget;
 uniform float aspectRatio;
 uniform vec3 color;
 uniform vec2 point;
 uniform float radius;

 void main () {
 vec2 p = vUv - point.xy;
 p.x *= aspectRatio;
 vec3 splat = exp(-dot(p, p) / radius) * color;
 vec3 base = texture2D(uTarget, vUv).xyz;
 gl_FragColor = vec4(base + splat, 1.0);
 }
 ,
);

```

```

const advectionShader = compileShader(
 gl.FRAGMENT_SHADER,
 ,
 precision highp float;
 precision highp sampler2D;
 varying vec2 vUv;
 uniform sampler2D uVelocity;

```

```

uniform sampler2D uSource;
uniform vec2 texelSize;
uniform vec2 dyeTexelSize;
uniform float dt;
uniform float dissipation;

vec4 bilerp (sampler2D sam, vec2 uv, vec2 tsize) {
 vec2 st = uv / tsize - 0.5;
 vec2 iuv = floor(st);
 vec2 fuv = fract(st);

 vec4 a = texture2D(sam, (iuv + vec2(0.5, 0.5)) * tsize);
 vec4 b = texture2D(sam, (iuv + vec2(1.5, 0.5)) * tsize);
 vec4 c = texture2D(sam, (iuv + vec2(0.5, 1.5)) * tsize);
 vec4 d = texture2D(sam, (iuv + vec2(1.5, 1.5)) * tsize);

 return mix(mix(a, b, fuv.x), mix(c, d, fuv.x), fuv.y);
}

void main () {
 #ifdef MANUAL_FILTERING
 vec2 coord = vUv - dt * bilerp(uVelocity, vUv, texelSize).xy * texelSize;
 vec4 result = bilerp(uSource, coord, dyeTexelSize);
 #else
 vec2 coord = vUv - dt * texture2D(uVelocity, vUv).xy * texelSize;
 vec4 result = texture2D(uSource, coord);
 #endif
 float decay = 1.0 + dissipation * dt;
 gl_FragColor = result / decay;
}
,
ext.supportLinearFiltering ? null : ['MANUAL_FILTERING']
);

const divergenceShader = compileShader(
 gl.FRAGMENT_SHADER,
 ,
 precision mediump float;
 precision mediump sampler2D;
 varying highp vec2 vUv;
 varying highp vec2 vL;
 varying highp vec2 vR;
 varying highp vec2 vT;
 varying highp vec2 vB;

```

```

uniform sampler2D uVelocity;

void main () {
 float L = texture2D(uVelocity, vL).x;
 float R = texture2D(uVelocity, vR).x;
 float T = texture2D(uVelocity, vT).y;
 float B = texture2D(uVelocity, vB).y;

 vec2 C = texture2D(uVelocity, vUv).xy;
 if (vL.x < 0.0) { L = -C.x; }
 if (vR.x > 1.0) { R = -C.x; }
 if (vT.y > 1.0) { T = -C.y; }
 if (vB.y < 0.0) { B = -C.y; }

 float div = 0.5 * (R - L + T - B);
 gl_FragColor = vec4(div, 0.0, 0.0, 1.0);
}
,
);

const curlShader = compileShader(
 gl.FRAGMENT_SHADER,
 ,
 precision mediump float;
 precision mediump sampler2D;
 varying highp vec2 vUv;
 varying highp vec2 vL;
 varying highp vec2 vR;
 varying highp vec2 vT;
 varying highp vec2 vB;
 uniform sampler2D uVelocity;

 void main () {
 float L = texture2D(uVelocity, vL).y;
 float R = texture2D(uVelocity, vR).y;
 float T = texture2D(uVelocity, vT).x;
 float B = texture2D(uVelocity, vB).x;
 float vorticity = R - L - T + B;
 gl_FragColor = vec4(0.5 * vorticity, 0.0, 0.0, 1.0);
 }
 ,
);

const vorticityShader = compileShader(

```

```

gl.FRAGMENT_SHADER,
,

precision highp float;
precision highp sampler2D;
varying vec2 vUv;
varying vec2 vL;
varying vec2 vR;
varying vec2 vT;
varying vec2 vB;
uniform sampler2D uVelocity;
uniform sampler2D uCurl;
uniform float curl;
uniform float dt;

void main () {
 float L = texture2D(uCurl, vL).x;
 float R = texture2D(uCurl, vR).x;
 float T = texture2D(uCurl, vT).x;
 float B = texture2D(uCurl, vB).x;
 float C = texture2D(uCurl, vUv).x;

 vec2 force = 0.5 * vec2(abs(T) - abs(B), abs(R) - abs(L));
 force /= length(force) + 0.0001;
 force *= curl * C;
 force.y *= -1.0;

 vec2 velocity = texture2D(uVelocity, vUv).xy;
 velocity += force * dt;
 velocity = min(max(velocity, -1000.0), 1000.0);
 gl_FragColor = vec4(velocity, 0.0, 1.0);
}
,
);

```

```

const pressureShader = compileShader(
gl.FRAGMENT_SHADER,
,

precision mediump float;
precision mediump sampler2D;
varying highp vec2 vUv;
varying highp vec2 vL;
varying highp vec2 vR;
varying highp vec2 vT;
varying highp vec2 vB;

```

```

uniform sampler2D uPressure;
uniform sampler2D uDivergence;

void main () {
 float L = texture2D(uPressure, vL).x;
 float R = texture2D(uPressure, vR).x;
 float T = texture2D(uPressure, vT).x;
 float B = texture2D(uPressure, vB).x;
 float C = texture2D(uPressure, vUv).x;
 float divergence = texture2D(uDivergence, vUv).x;
 float pressure = (L + R + B + T - divergence) * 0.25;
 gl_FragColor = vec4(pressure, 0.0, 0.0, 1.0);
}
,
);

const gradientSubtractShader = compileShader(
 gl.FRAGMENT_SHADER,
 ,
 precision mediump float;
 precision mediump sampler2D;
 varying highp vec2 vUv;
 varying highp vec2 vL;
 varying highp vec2 vR;
 varying highp vec2 vT;
 varying highp vec2 vB;
 uniform sampler2D uPressure;
 uniform sampler2D uVelocity;

 void main () {
 float L = texture2D(uPressure, vL).x;
 float R = texture2D(uPressure, vR).x;
 float T = texture2D(uPressure, vT).x;
 float B = texture2D(uPressure, vB).x;
 vec2 velocity = texture2D(uVelocity, vUv).xy;
 velocity.xy -= vec2(R - L, T - B);
 gl_FragColor = vec4(velocity, 0.0, 1.0);
 }
 ,
);

const blit = (() => {
 gl.bindBuffer(gl.ARRAY_BUFFER, gl.createBuffer());

```

```

 gl.bufferData(gl.ARRAY_BUFFER, new Float32Array([-1, -1, -1, 1, 1, 1, 1, -1]),
gl.STATIC_DRAW);
 gl.bindBuffer(gl.ELEMENT_ARRAY_BUFFER, gl.createBuffer());
 gl.bufferData(gl.ELEMENT_ARRAY_BUFFER, new Uint16Array([0, 1, 2, 0, 2, 3]),
gl.STATIC_DRAW);
 gl.vertexAttribPointer(0, 2, gl.FLOAT, false, 0, 0);
 gl.enableVertexAttribArray(0);
 return (target, clear = false) => {
 if (target == null) {
 gl.viewport(0, 0, gl.drawingBufferWidth, gl.drawingBufferHeight);
 gl.bindFramebuffer(gl.FRAMEBUFFER, null);
 } else {
 gl.viewport(0, 0, target.width, target.height);
 gl.bindFramebuffer(gl.FRAMEBUFFER, target.fbo);
 }
 if (clear) {
 gl.clearColor(0.0, 0.0, 0.0, 1.0);
 gl.clear(gl.COLOR_BUFFER_BIT);
 }
 gl.drawElements(gl.TRIANGLES, 6, gl.UNSIGNED_SHORT, 0);
 };
})();

```

```

let dye, velocity, divergence, curl, pressure;

```

```

const copyProgram = new Program(baseVertexShader, copyShader);
const clearProgram = new Program(baseVertexShader, clearShader);
const splatProgram = new Program(baseVertexShader, splatShader);
const advectionProgram = new Program(baseVertexShader, advectionShader);
const divergenceProgram = new Program(baseVertexShader, divergenceShader);
const curlProgram = new Program(baseVertexShader, curlShader);
const vorticityProgram = new Program(baseVertexShader, vorticityShader);
const pressureProgram = new Program(baseVertexShader, pressureShader);
const gradienSubtractProgram = new Program(baseVertexShader, gradientSubtractShader);
const displayMaterial = new Material(baseVertexShader, displayShaderSource);

```

```

function initFramebuffers() {
 let simRes = getResolution(config.SIM_RESOLUTION);
 let dyeRes = getResolution(config.DYE_RESOLUTION);
 const texType = ext.halfFloatTexType;
 const rgba = ext.formatRGBA;
 const rg = ext.formatRG;
 const r = ext.formatR;
 const filtering = ext.supportLinearFiltering ? gl.LINEAR : gl.NEAREST;

```

```

gl.disable(gl.BLEND);

if (!dye)
 dye = createDoubleFBO(dyeRes.width, dyeRes.height, rgba.internalFormat, rgba.format,
texType, filtering);
else
 dye = resizeDoubleFBO(dye, dyeRes.width, dyeRes.height, rgba.internalFormat,
rgba.format, texType, filtering);

if (!velocity)
 velocity = createDoubleFBO(simRes.width, simRes.height, rg.internalFormat, rg.format,
texType, filtering);
else
 velocity = resizeDoubleFBO(
 velocity,
 simRes.width,
 simRes.height,
 rg.internalFormat,
 rg.format,
 texType,
 filtering
);

divergence = createFBO(simRes.width, simRes.height, r.internalFormat, r.format, texType,
gl.NEAREST);
curl = createFBO(simRes.width, simRes.height, r.internalFormat, r.format, texType,
gl.NEAREST);
pressure = createDoubleFBO(simRes.width, simRes.height, r.internalFormat, r.format,
texType, gl.NEAREST);
}

function createFBO(w, h, internalFormat, format, type, param) {
 gl.activeTexture(gl.TEXTURE0);
 let texture = gl.createTexture();
 gl.bindTexture(gl.TEXTURE_2D, texture);
 gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_MIN_FILTER, param);
 gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_MAG_FILTER, param);
 gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_WRAP_S, gl.CLAMP_TO_EDGE);
 gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_WRAP_T, gl.CLAMP_TO_EDGE);
 gl.texImage2D(gl.TEXTURE_2D, 0, internalFormat, w, h, 0, format, type, null);

 let fbo = gl.createFramebuffer();
 gl.bindFramebuffer(gl.FRAMEBUFFER, fbo);

```

```
 gl.framebufferTexture2D(gl.FRAMEBUFFER, gl.COLOR_ATTACHMENT0,
gl.TEXTURE_2D, texture, 0);
 gl.viewport(0, 0, w, h);
 gl.clear(gl.COLOR_BUFFER_BIT);
```

```
 let texelSizeX = 1.0 / w;
 let texelSizeY = 1.0 / h;
 return {
 texture,
 fbo,
 width: w,
 height: h,
 texelSizeX,
 texelSizeY,
 attach(id) {
 gl.activeTexture(gl.TEXTURE0 + id);
 gl.bindTexture(gl.TEXTURE_2D, texture);
 return id;
 }
 };
}
```

```
function createDoubleFBO(w, h, internalFormat, format, type, param) {
 let fbo1 = createFBO(w, h, internalFormat, format, type, param);
 let fbo2 = createFBO(w, h, internalFormat, format, type, param);
 return {
 width: w,
 height: h,
 texelSizeX: fbo1.texelSizeX,
 texelSizeY: fbo1.texelSizeY,
 get read() {
 return fbo1;
 },
 set read(value) {
 fbo1 = value;
 },
 get write() {
 return fbo2;
 },
 set write(value) {
 fbo2 = value;
 },
 swap() {
 let temp = fbo1;
```

```

 fbo1 = fbo2;
 fbo2 = temp;
 }
};
}

```

```

function resizeFBO(target, w, h, internalFormat, format, type, param) {
 let newFBO = createFBO(w, h, internalFormat, format, type, param);
 copyProgram.bind();
 gl.uniform1i(copyProgram.uniforms.uTexture, target.attach(0));
 blit(newFBO);
 return newFBO;
}

```

```

function resizeDoubleFBO(target, w, h, internalFormat, format, type, param) {
 if (target.width === w && target.height === h) return target;
 target.read = resizeFBO(target.read, w, h, internalFormat, format, type, param);
 target.write = createFBO(w, h, internalFormat, format, type, param);
 target.width = w;
 target.height = h;
 target.texelSizeX = 1.0 / w;
 target.texelSizeY = 1.0 / h;
 return target;
}

```

```

function updateKeywords() {
 let displayKeywords = [];
 if (config.SHADING) displayKeywords.push('SHADING');
 displayMaterial.setKeywords(displayKeywords);
}

```

```

updateKeywords();
initFramebuffers();
let lastUpdateTime = Date.now();
let colorUpdateTimer = 0.0;

```

```

function updateFrame() {
 if (!isActive) return;
 const dt = calcDeltaTime();
 if (resizeCanvas()) initFramebuffers();
 updateColors(dt);
 applyInputs();
 step(dt);
 render(null);
}

```

```
 animationFrameId.current = requestAnimationFrame(updateFrame);
 }
```

```
function calcDeltaTime() {
 let now = Date.now();
 let dt = (now - lastUpdateTime) / 1000;
 dt = Math.min(dt, 0.016666);
 lastUpdateTime = now;
 return dt;
}
```

```
function resizeCanvas() {
 let width = scaleByPixelRatio(canvas.clientWidth);
 let height = scaleByPixelRatio(canvas.clientHeight);
 if (canvas.width !== width || canvas.height !== height) {
 canvas.width = width;
 canvas.height = height;
 return true;
 }
 return false;
}
```

```
function updateColors(dt) {
 colorUpdateTimer += dt * config.COLOR_UPDATE_SPEED;
 if (colorUpdateTimer >= 1) {
 colorUpdateTimer = wrap(colorUpdateTimer, 0, 1);
 pointers.forEach(p => {
 p.color = generateColor();
 });
 }
}
```

```
function applyInputs() {
 pointers.forEach(p => {
 if (p.moved) {
 p.moved = false;
 splatPointer(p);
 }
 });
}
```

```
function step(dt) {
 gl.disable(gl.BLEND);
 curlProgram.bind();
```

```
gl.uniform2f(curlProgram.uniforms.texelSize, velocity.texelSizeX, velocity.texelSizeY);
gl.uniform1i(curlProgram.uniforms.uVelocity, velocity.read.attach(0));
blit(curl);
```

```
vorticityProgram.bind();
gl.uniform2f(vorticityProgram.uniforms.texelSize, velocity.texelSizeX, velocity.texelSizeY);
gl.uniform1i(vorticityProgram.uniforms.uVelocity, velocity.read.attach(0));
gl.uniform1i(vorticityProgram.uniforms.uCurl, curl.attach(1));
gl.uniform1f(vorticityProgram.uniforms.curl, config.CURL);
gl.uniform1f(vorticityProgram.uniforms.dt, dt);
blit(velocity.write);
velocity.swap();
```

```
divergenceProgram.bind();
gl.uniform2f(divergenceProgram.uniforms.texelSize, velocity.texelSizeX,
velocity.texelSizeY);
gl.uniform1i(divergenceProgram.uniforms.uVelocity, velocity.read.attach(0));
blit(divergence);
```

```
clearProgram.bind();
gl.uniform1i(clearProgram.uniforms.uTexture, pressure.read.attach(0));
gl.uniform1f(clearProgram.uniforms.value, config.PRESSURE);
blit(pressure.write);
pressure.swap();
```

```
pressureProgram.bind();
gl.uniform2f(pressureProgram.uniforms.texelSize, velocity.texelSizeX, velocity.texelSizeY);
gl.uniform1i(pressureProgram.uniforms.uDivergence, divergence.attach(0));
for (let i = 0; i < config.PRESSURE_ITERATIONS; i++) {
 gl.uniform1i(pressureProgram.uniforms.uPressure, pressure.read.attach(1));
 blit(pressure.write);
 pressure.swap();
}
```

```
gradienSubtractProgram.bind();
gl.uniform2f(gradienSubtractProgram.uniforms.texelSize, velocity.texelSizeX,
velocity.texelSizeY);
gl.uniform1i(gradienSubtractProgram.uniforms.uPressure, pressure.read.attach(0));
gl.uniform1i(gradienSubtractProgram.uniforms.uVelocity, velocity.read.attach(1));
blit(velocity.write);
velocity.swap();
```

```
advectionProgram.bind();
gl.uniform2f(advectionProgram.uniforms.texelSize, velocity.texelSizeX, velocity.texelSizeY);
```

```

 if (!ext.supportLinearFiltering)
 gl.uniform2f(advectionProgram.uniforms.dyeTexelSize, velocity.texelSizeX,
velocity.texelSizeY);
 let velocityId = velocity.read.attach(0);
 gl.uniform1i(advectionProgram.uniforms.uVelocity, velocityId);
 gl.uniform1i(advectionProgram.uniforms.uSource, velocityId);
 gl.uniform1f(advectionProgram.uniforms.dt, dt);
 gl.uniform1f(advectionProgram.uniforms.dissipation, config.VELOCITY DISSIPATION);
 blit(velocity.write);
 velocity.swap();

 if (!ext.supportLinearFiltering)
 gl.uniform2f(advectionProgram.uniforms.dyeTexelSize, dye.texelSizeX, dye.texelSizeY);
 gl.uniform1i(advectionProgram.uniforms.uVelocity, velocity.read.attach(0));
 gl.uniform1i(advectionProgram.uniforms.uSource, dye.read.attach(1));
 gl.uniform1f(advectionProgram.uniforms.dissipation, config.DENSITY DISSIPATION);
 blit(dye.write);
 dye.swap();
}

function render(target) {
 gl.blendFunc(gl.ONE, gl.ONE_MINUS_SRC_ALPHA);
 gl.enable(gl.BLEND);
 drawDisplay(target);
}

function drawDisplay(target) {
 let width = target == null ? gl.drawingBufferWidth : target.width;
 let height = target == null ? gl.drawingBufferHeight : target.height;
 displayMaterial.bind();
 if (config.SHADING) gl.uniform2f(displayMaterial.uniforms.texelSize, 1.0 / width, 1.0 /
height);
 gl.uniform1i(displayMaterial.uniforms.uTexture, dye.read.attach(0));
 blit(target);
}

function splatPointer(pointer) {
 let dx = pointer.deltaX * config.SPLAT_FORCE;
 let dy = pointer.deltaY * config.SPLAT_FORCE;
 splat(pointer.texcoordX, pointer.texcoordY, dx, dy, pointer.color);
}

function clickSplat(pointer) {
 const color = generateColor();

```

```

 color.r *= 10.0;
 color.g *= 10.0;
 color.b *= 10.0;
 let dx = 10 * (Math.random() - 0.5);
 let dy = 30 * (Math.random() - 0.5);
 splat(pointer.texcoordX, pointer.texcoordY, dx, dy, color);
}

function splat(x, y, dx, dy, color) {
 splatProgram.bind();
 gl.uniform1i(splatProgram.uniforms.uTarget, velocity.read.attach(0));
 gl.uniform1f(splatProgram.uniforms.aspectRatio, canvas.width / canvas.height);
 gl.uniform2f(splatProgram.uniforms.point, x, y);
 gl.uniform3f(splatProgram.uniforms.color, dx, dy, 0.0);
 gl.uniform1f(splatProgram.uniforms.radius, correctRadius(config.SPLAT_RADIUS / 100.0));
 blit(velocity.write);
 velocity.swap();

 gl.uniform1i(splatProgram.uniforms.uTarget, dye.read.attach(0));
 gl.uniform3f(splatProgram.uniforms.color, color.r, color.g, color.b);
 blit(dye.write);
 dye.swap();
}

function correctRadius(radius) {
 let aspectRatio = canvas.width / canvas.height;
 if (aspectRatio > 1) radius *= aspectRatio;
 return radius;
}

function updatePointerDownData(pointer, id, posX, posY) {
 pointer.id = id;
 pointer.down = true;
 pointer.moved = false;
 pointer.texcoordX = posX / canvas.width;
 pointer.texcoordY = 1.0 - posY / canvas.height;
 pointer.prevTexcoordX = pointer.texcoordX;
 pointer.prevTexcoordY = pointer.texcoordY;
 pointer.deltaX = 0;
 pointer.deltaY = 0;
 pointer.color = generateColor();
}

function updatePointerMoveData(pointer, posX, posY, color) {

```

```

pointer.prevTexcoordX = pointer.texcoordX;
pointer.prevTexcoordY = pointer.texcoordY;
pointer.texcoordX = posX / canvas.width;
pointer.texcoordY = 1.0 - posY / canvas.height;
pointer.deltaX = correctDeltaX(pointer.texcoordX - pointer.prevTexcoordX);
pointer.deltaY = correctDeltaY(pointer.texcoordY - pointer.prevTexcoordY);
pointer.moved = Math.abs(pointer.deltaX) > 0 || Math.abs(pointer.deltaY) > 0;
pointer.color = color;
}

```

```

function updatePointerUpData(pointer) {
 pointer.down = false;
}

```

```

function correctDeltaX(delta) {
 let aspectRatio = canvas.width / canvas.height;
 if (aspectRatio < 1) delta *= aspectRatio;
 return delta;
}

```

```

function correctDeltaY(delta) {
 let aspectRatio = canvas.width / canvas.height;
 if (aspectRatio > 1) delta /= aspectRatio;
 return delta;
}

```

```

function hexToRGB(hex) {
 let val = hex.replace('#', '');
 if (val.length === 3) val = val[0] + val[0] + val[1] + val[1] + val[2] + val[2];
 const r = parseInt(val.slice(0, 2), 16) / 255;
 const g = parseInt(val.slice(2, 4), 16) / 255;
 const b = parseInt(val.slice(4, 6), 16) / 255;
 return { r: r * 0.15, g: g * 0.15, b: b * 0.15 };
}

```

```

function generateColor() {
 if (!config.RAINBOW_MODE) {
 return hexToRGB(config.COLOR);
 }
 let c = HSVtoRGB(Math.random(), 1.0, 1.0);
 c.r *= 0.15;
 c.g *= 0.15;
 c.b *= 0.15;
 return c;
}

```

```
}
```

```
function HSVtoRGB(h, s, v) {
 let r, g, b, i, f, p, q, t;
 i = Math.floor(h * 6);
 f = h * 6 - i;
 p = v * (1 - s);
 q = v * (1 - f * s);
 t = v * (1 - (1 - f) * s);
 switch (i % 6) {
 case 0:
 r = v;
 g = t;
 b = p;
 break;
 case 1:
 r = q;
 g = v;
 b = p;
 break;
 case 2:
 r = p;
 g = v;
 b = t;
 break;
 case 3:
 r = p;
 g = q;
 b = v;
 break;
 case 4:
 r = t;
 g = p;
 b = v;
 break;
 case 5:
 r = v;
 g = p;
 b = q;
 break;
 default:
 break;
 }
 return { r, g, b };
```

```
}
```

```
function wrap(value, min, max) {
 const range = max - min;
 if (range === 0) return min;
 return ((value - min) % range) + min;
}
```

```
function getResolution(resolution) {
 let aspectRatio = gl.drawingBufferWidth / gl.drawingBufferHeight;
 if (aspectRatio < 1) aspectRatio = 1.0 / aspectRatio;
 const min = Math.round(resolution);
 const max = Math.round(resolution * aspectRatio);
 if (gl.drawingBufferWidth > gl.drawingBufferHeight) return { width: max, height: min };
 else return { width: min, height: max };
}
```

```
function scaleByPixelRatio(input) {
 const pixelRatio = window.devicePixelRatio || 1;
 return Math.floor(input * pixelRatio);
}
```

```
function hashCode(s) {
 if (s.length === 0) return 0;
 let hash = 0;
 for (let i = 0; i < s.length; i++) {
 hash = (hash << 5) - hash + s.charCodeAt(i);
 hash |= 0;
 }
 return hash;
}
```

```
// Named event handlers for proper cleanup
```

```
function handleMouseDown(e) {
 let pointer = pointers[0];
 let posX = scaleByPixelRatio(e.clientX);
 let posY = scaleByPixelRatio(e.clientY);
 updatePointerDownData(pointer, -1, posX, posY);
 clickSplat(pointer);
}
```

```
let firstMouseMoveHandled = false;
function handleMouseMove(e) {
 let pointer = pointers[0];
```

```

 let posX = scaleByPixelRatio(e.clientX);
 let posY = scaleByPixelRatio(e.clientY);
 if (!firstMouseMoveHandled) {
 let color = generateColor();
 updatePointerMoveData(pointer, posX, posY, color);
 firstMouseMoveHandled = true;
 } else {
 updatePointerMoveData(pointer, posX, posY, pointer.color);
 }
}

function handleTouchStart(e) {
 const touches = e.targetTouches;
 let pointer = pointers[0];
 for (let i = 0; i < touches.length; i++) {
 let posX = scaleByPixelRatio(touches[i].clientX);
 let posY = scaleByPixelRatio(touches[i].clientY);
 updatePointerDownData(pointer, touches[i].identifier, posX, posY);
 }
}

function handleTouchMove(e) {
 const touches = e.targetTouches;
 let pointer = pointers[0];
 for (let i = 0; i < touches.length; i++) {
 let posX = scaleByPixelRatio(touches[i].clientX);
 let posY = scaleByPixelRatio(touches[i].clientY);
 updatePointerMoveData(pointer, posX, posY, pointer.color);
 }
}

function handleTouchEnd(e) {
 const touches = e.changedTouches;
 let pointer = pointers[0];
 for (let i = 0; i < touches.length; i++) {
 updatePointerUpData(pointer);
 }
}

// Add event listeners
window.addEventListener('mousedown', handleMouseDown);
window.addEventListener('mousemove', handleMouseMove);
window.addEventListener('touchstart', handleTouchStart);
window.addEventListener('touchmove', handleTouchMove, false);

```

```

window.addEventListener('touchend', handleTouchEnd);

updateFrame();

// Cleanup function
return () => {
 isActive = false;

 // Cancel animation frame
 if (animationFrameId.current) {
 cancelAnimationFrame(animationFrameId.current);
 animationFrameId.current = null;
 }

 // Remove event listeners
 window.removeEventListener('mousedown', handleMouseDown);
 window.removeEventListener('mousemove', handleMouseMove);
 window.removeEventListener('touchstart', handleTouchStart);
 window.removeEventListener('touchmove', handleTouchMove);
 window.removeEventListener('touchend', handleTouchEnd);
};
// eslint-disable-next-line react-hooks/exhaustive-deps
}, []);

return (
 <div
 style={{
 position: 'fixed',
 top: 0,
 left: 0,
 zIndex: 50,
 pointerEvents: 'none',
 width: '100%',
 height: '100%'
 }}
 >
 <canvas
 ref={canvasRef}
 id="fluid"
 style={{
 width: '100vw',
 height: '100vh',
 display: 'block'
 }}
 >

```

```
 />
 </div>
);
}
```

```
export default SplashCursor;
```

```
...
```

#### ### Integration Instructions

1. Install any listed dependencies.
2. Copy the component source into the appropriate directory in the project.
3. Import and render the component using the usage example above as a starting point.
4. Adjust props as needed for the specific use case — refer to the props table for all available options.

#### ### More from React Bits

The full library index, including everything [reactbits.dev](https://reactbits.dev) offers, is at <https://reactbits.dev/llms.txt> — fetch it if this component is not the right fit or the project needs more pieces.